Hyperdimensional Computing Labs

UNDERSTANDING HYPERDIMENSIONAL COMPUTING (HDC) THROUGH TELECOM INCIDENTS

Signal, Context & Memory

An empirical study of telecom incident triage



Prashanth Rao

Cofounder, HDC Labs
AI Engineering & Research Lead



David Hughes

Founder, HDC Labs
Principal AI Solutions Architect



Synthetic network, records and reviews. Only Toronto street geometry is real, and obtained from open map data.

Hyperdimensional Computing Labs

Evaluating HDC for telecom incident triage

We study a phone and its connected network through three observations over time.

01



Representation

What information must we preserve?

Measurements, connections and their order in time.

02

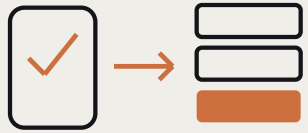


Retrieval

Which earlier cases look similar?

Comparable cases, linked to the original records.

03



Learning

What changes after a review?

Reviewed examples update memory for later decisions.

04



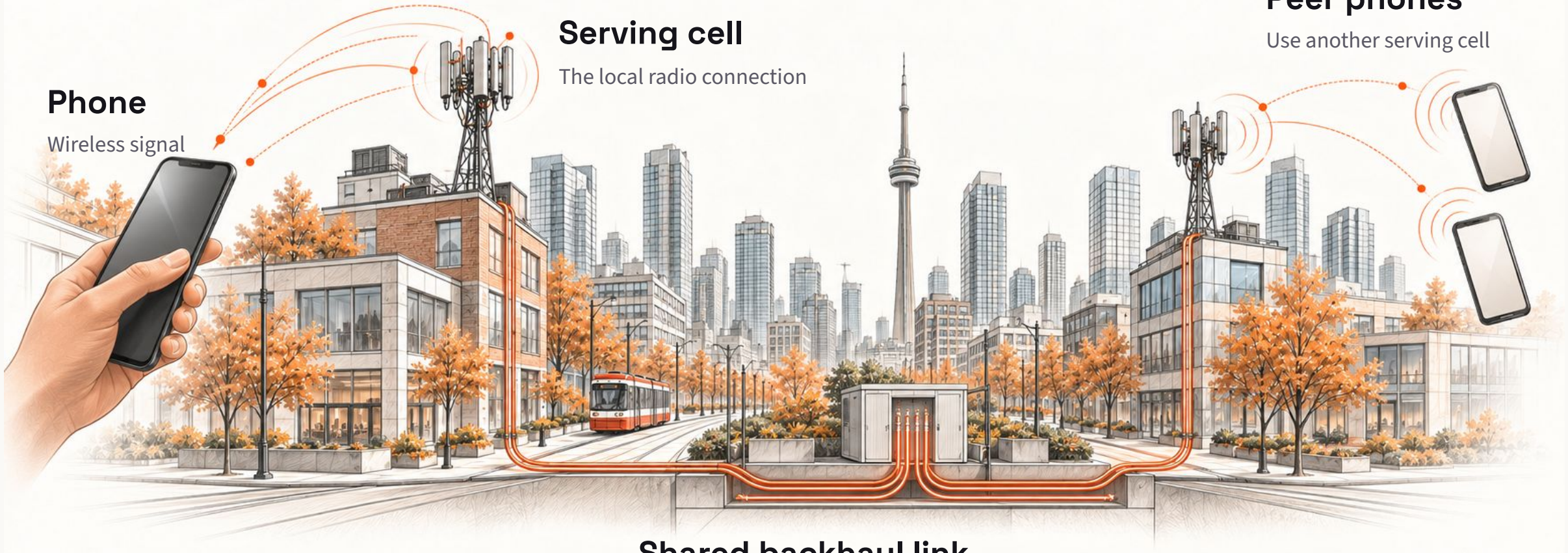
Resources

What does each step cost?

Time to predict and learn; space to store incidents.

Four themes for understanding HDC

A phone depends on a connected network



Phone

Wireless signal

Serving cell

The local radio connection

Peer phones

Use another serving cell

Shared backhaul link

Carries traffic from both cells

Peer measurements add context about a shared dependency

Hyperdimensional Computing Labs

Inside an episode: phone and network readings over time

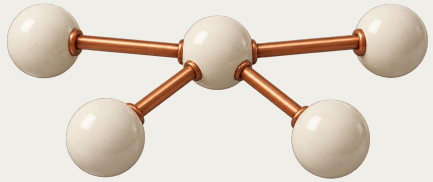
	0 seconds	20 seconds	40 seconds
			
Signal power	−80.6 dBm	−91.7 dBm	−110.6 dBm
Backhaul delivery improves Fewer packets lost (%)	1.24%	0.95%	0.26%

dBm measures received signal power (decibels relative to 1 milliwatt).
More negative means a **weaker** signal: −110.6 dBm is weaker than −80.6 dBm.

HDC adds similarity and learning to graph structure

The graph records connections; our encoder carries their meaning into a shared vector space.

EXPLICIT GRAPH



Nodes and edges

The data model and traversal specify which relationships to follow. Similarity and learning need added methods.

Exact connections and source evidence

HDC REPRESENTATION



Structure within a hypervector

Encode selected roles, values and order together, then compare whole episodes by cosine similarity.

Distributed encoding, assigned roles

HDC LEARNING



Update memory with class prototypes

Each reviewed episode updates its class's prototype. The encoder stays fixed; no backpropagation is needed.

Simple vector update operations

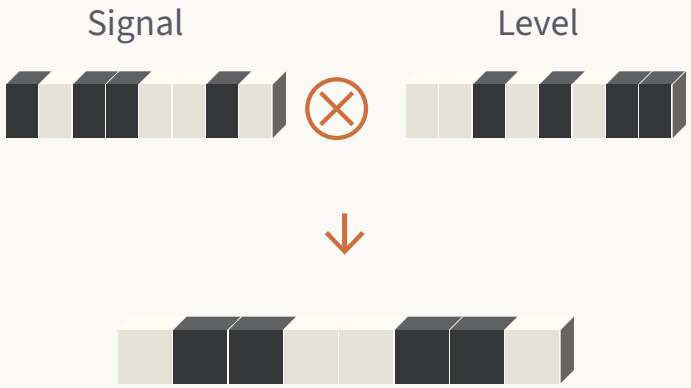
While a typical GNN learns weights through gradient descent, our HDC model learns by **updating a class memory.**

The three operators of HDC can represent a graph path

Phone → Cell → Link

01 Bind ⊗

Represent each node's attributes

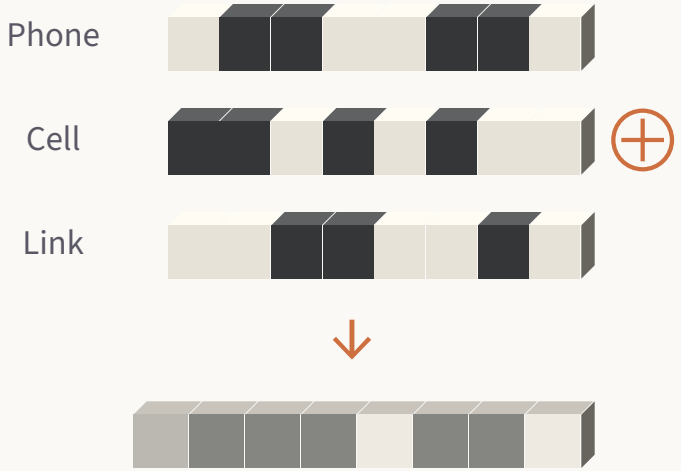


Phone hypervector

Bind roles to values;
bundle attributes within each node

02 Bundle ⊕

Bring the path's nodes together

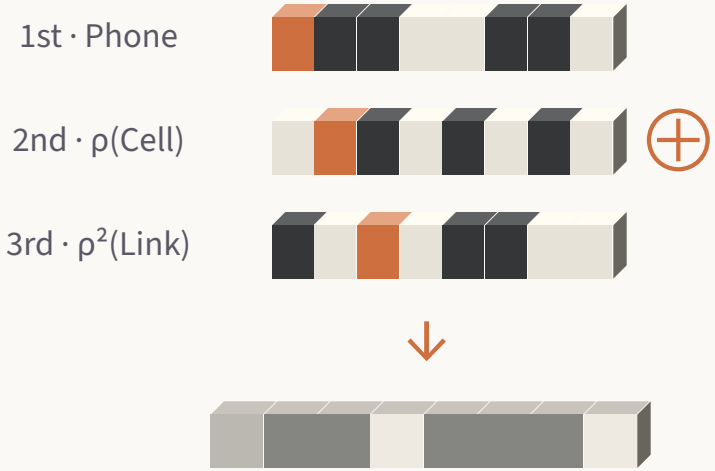


Path contents

Bundling alone loses order

03 Permute ρ

Give the sequence an order



Ordered path

Shift by position, then combine

The same construction extends to longer paths and typed edges

Hyperdimensional Computing Labs

Nearby numbers become similar hypervectors

We define dBm as **level hypervectors**, which places the measurements on an **ordered scale**.

−125 dBm — 32 levels across the chosen signal range — −65 dBm



MEASUREMENT → LEVEL

LEVEL HYPERVECTOR

−80.6 dBm

Level 23



Reference reading

−82.6 dBm

Level 22



Close value

Most coordinates stay the same

−110.6 dBm

Level 7



Farther value

More coordinates differ

Signal ⊗ Level 23

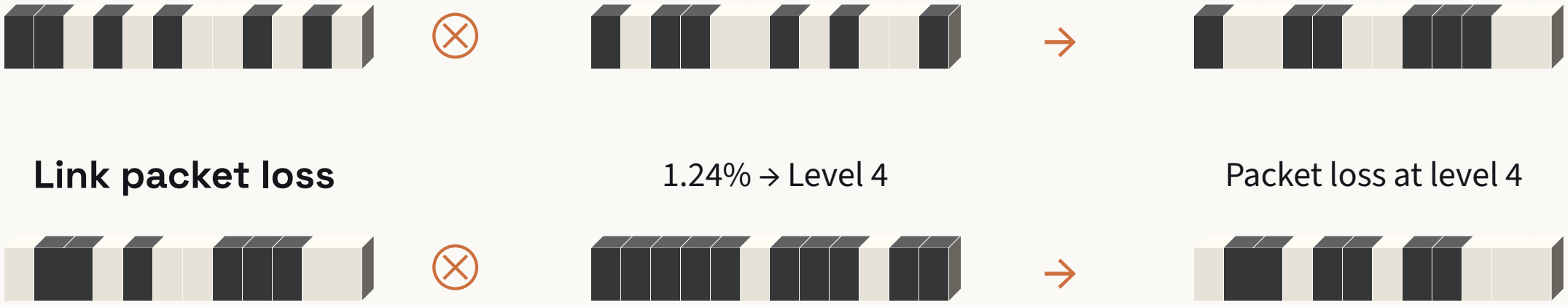
Binding gives the number its meaning within the node

Illustrative hypervector colours · Underlines mark changes from the reference · The real hypervectors have a dimensionality of 4,096

Binding keeps signal and packet loss distinct

A level carries magnitude; its role tells us which property it describes

PROPERTY ROLE	NUMERIC LEVEL	BOUND FACT
Phone signal	−80.6 dBm → Level 23	Signal at level 23
Link packet loss	1.24% → Level 4	Packet loss at level 4



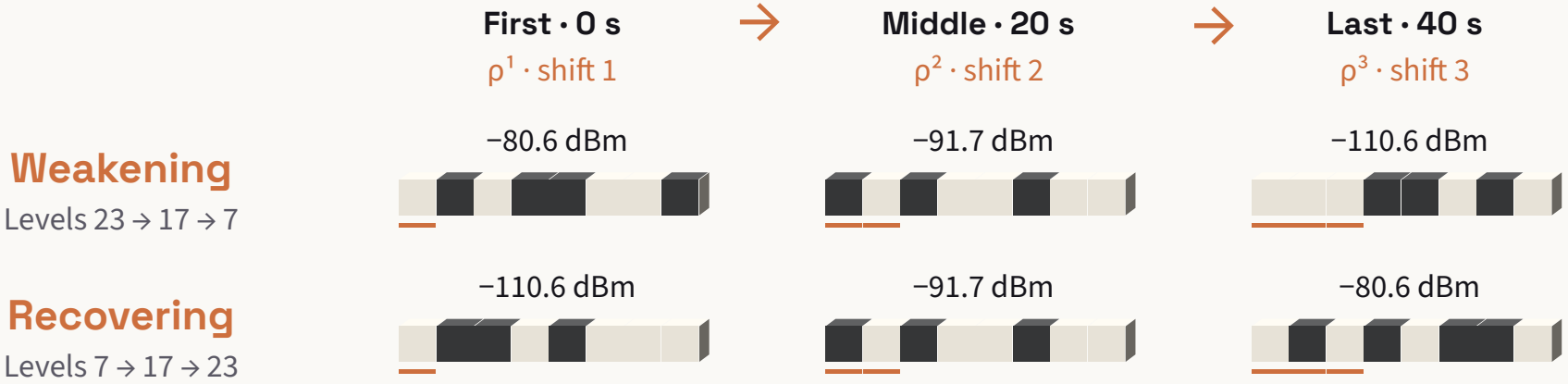
Bind each property to its level, then bundle the resulting facts

A shared role preserves similarity between nearby levels

Hyperdimensional Computing Labs

Permutation distinguishes deterioration from recovery

- Each reading at a particular dBm has its own hypervector
- We circularly shift its coordinates by 1, 2 or 3 places to mark whether it came first, second or third.



Before shifting: A = signal at level 23 B = signal at level 17 C = signal at level 7

Encoding	Weakening	Recovering
Without permutation (order is lost)	$A \oplus B \oplus C$	$C \oplus B \oplus A$ = same bundle
With permutation (order is preserved)	$\rho^1(A) \oplus \rho^2(B) \oplus \rho^3(C)$	$\rho^1(C) \oplus \rho^2(B) \oplus \rho^3(A)$

Unique readings identify what happened at each stage. Position shifts **preserve the order** in a sequence.

Shifts 1, 2, 3 element indices • The labels indicate positions in the path: first, middle, last (not elapsed time).

Nineteen contributions become one episode hypervector

Combine the phone's trajectory, connected measurements and handset model

3 phone measurements

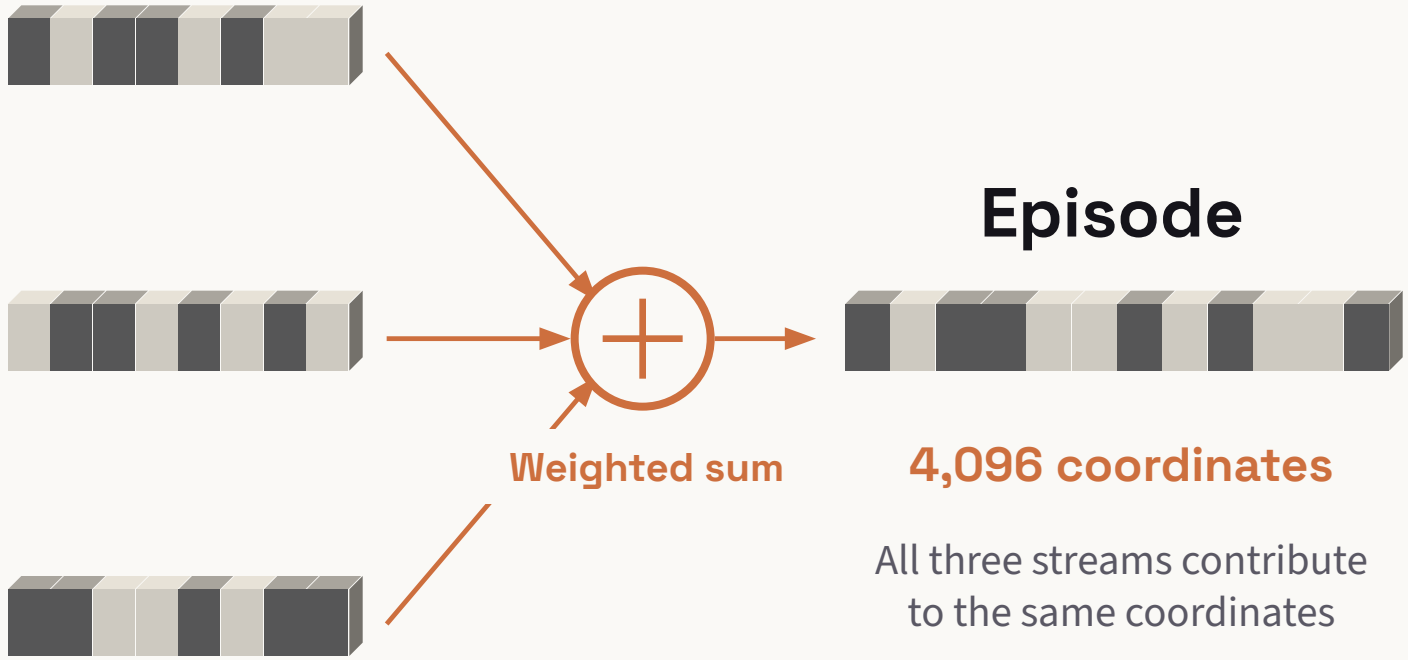
Signal at 0, 20 and 40 seconds

15 context readings

Cell load · link loss and latency
Peer signal and loss · at all three times

1 handset category

Device model, once per episode



More facts enrich the representation without making the hypervector longer

Numeric facts are already bound and shifted by observation position

Hyperdimensional Computing Labs

Lance tables support both vector search and graph traversal

LanceDB stores the records and hypervectors. lance-graph follows relationships defined by their IDs.

NODE TABLES

phones	Example value
phone_id	P17
handset	model_0
subscriber_id	S17
world_id	W1

Also: cells, links, observations

RELATIONSHIP TABLES

edges	Example value
edge_id (row ID)	R9
cell_id (source)	C4
link_id (target)	L2
valid_from_s	0
valid_to_s	86400

Cell → BACKHAULED_BY → Link

EPISODE HYPERVECTORS

episodes	Example value
episode_id	E7
source_ids	[01, 02, 03]
raw_vector	[0.6, -1.1, …]
vector	[0.01, -0.02, …]
manifest_json	"{terms: …}"

Vectors refer back to source records

lance-graph traverses the source records behind a retrieved episode

CYPHER · follow the relationship

```
MATCH (c:Cell)-[:BACKHAULED_BY]→(l:Link)
WHERE c.cell_id = 'C4'
RETURN l.link_id
```

SQL · count relationship rows per link

```
SELECT link_id, COUNT(*) AS connections
FROM edges
GROUP BY link_id
```


Illustrative IDs and abbreviated rows. The graph reuses the stored source tables; no separate persistent graph copy.

Find related episodes through associative search

Once we encode raw episode data as hypervectors, we can use similarity search to retrieve relevant earlier cases

QUERY EPISODE

Signal · Context · Order

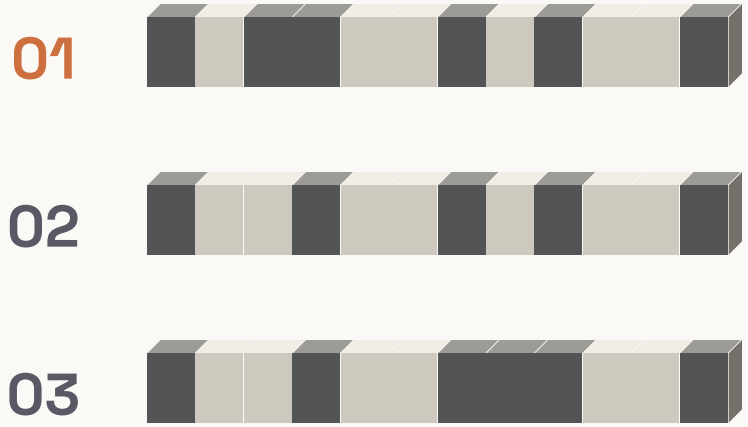


One holistic representation

Compare with memory

Rank by cosine similarity

SIMILAR PAST EPISODES



Most similar first

Like search with text embeddings

Compare vectors to retrieve related examples

Explicit structure and semantics

Measurements, role meanings, relationships and order

Hyperdimensional Computing Labs

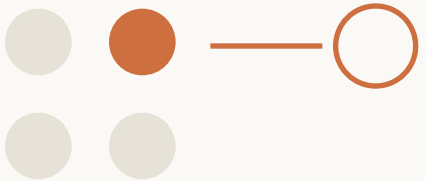
One representation supports search and learning



RETRIEVE EXAMPLES

What resembles this episode?

Find similar earlier cases and inspect the records behind each match.



LEARN FROM LABELLED EXAMPLES

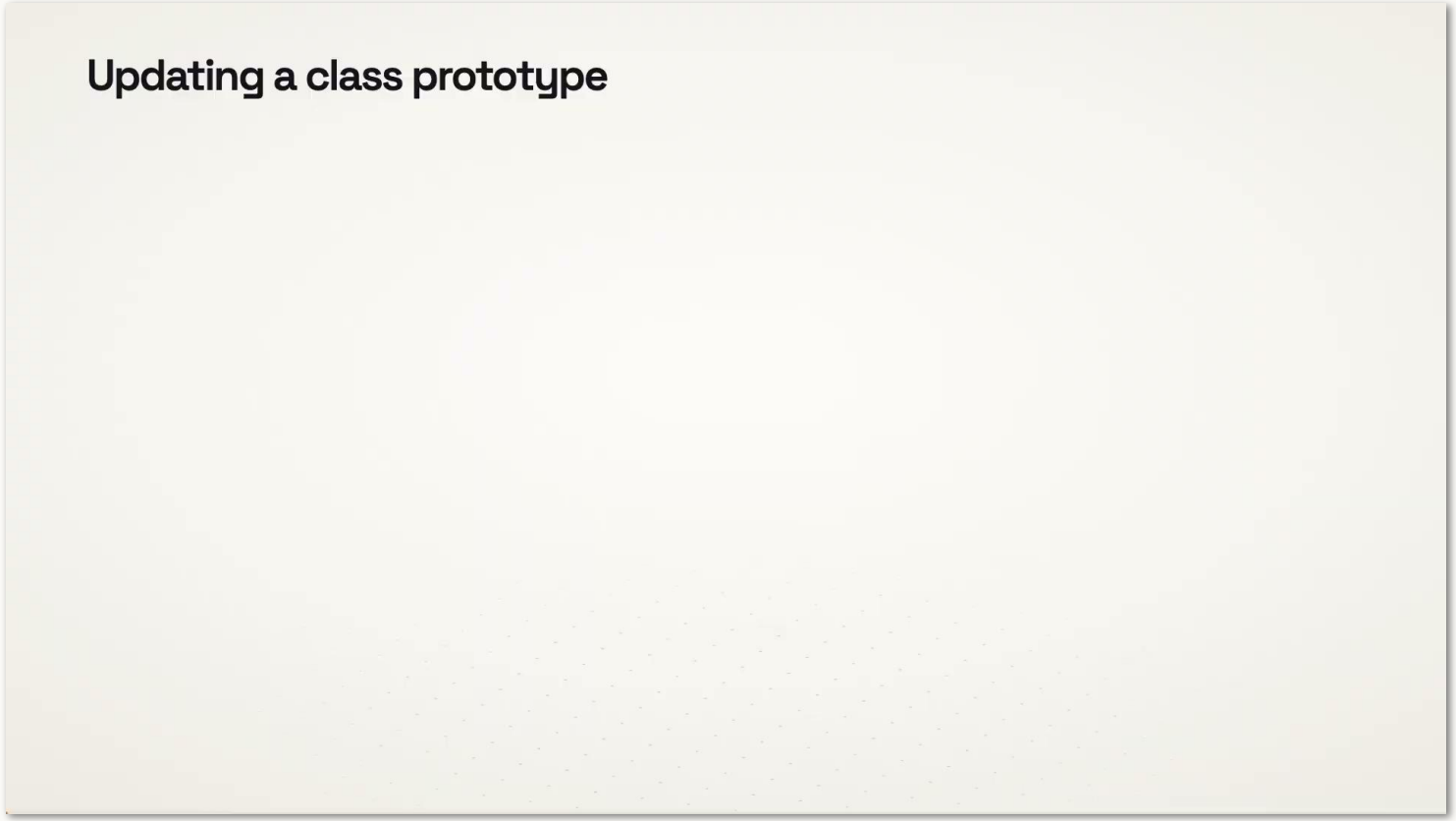
How does the system learn from new data?

Each reviewed episode updates class memory for future predictions.

Source records remain available for inspection alongside the hypervectors

A class memory grows with each labelled example

A prototype is built with examples of the same label, contributing to one fixed representation that can be updated



Build the class memory

Add together the hypervectors of examples with the same label.

Use its prototype

The direction of that sum is the class's representative hypervector.

Learn from another example

Add it to that class memory.
The prototype changes with it.

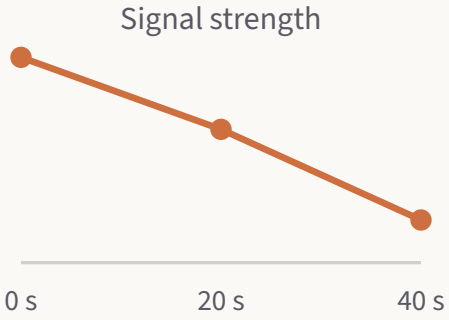
Predict a new example's class by finding the most similar prototype

Hyperdimensional Computing Labs

We build a separate memory for each incident class

Add each labelled episode's hypervector to the memory for its label

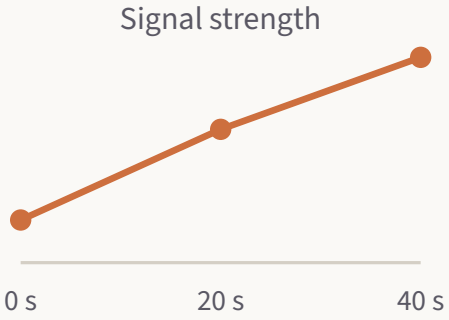
01



Deteriorating radio

The phone's signal gets weaker

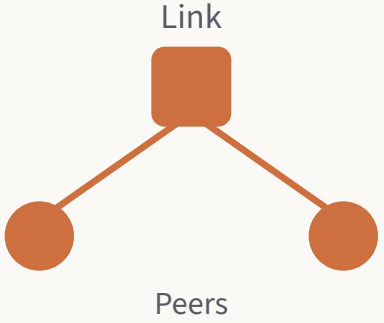
02



Transient recovery

The signal improves by the final observation

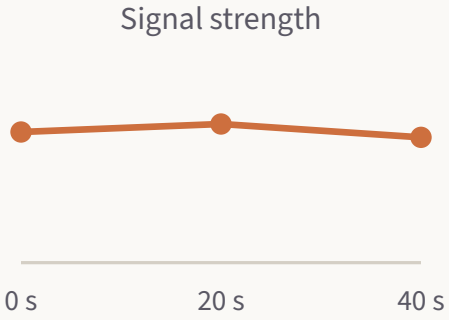
03



Shared impairment

Packet loss appears on the link and peer phones

04



Normal service

No major signal change or shared impairment

A deteriorating-radio example updates only the deteriorating-radio memory

How labelled episodes are added to class memory

Add the labelled episode's normalized hypervector to the deteriorating-radio class's running sum

Existing class memory



Sum of earlier examples

New labelled episode



Normalize to unit length

Updated class memory



Add coordinate by coordinate

Form the updated prototype

Normalize the new class sum.
Its direction represents the class.

Predict the next episode

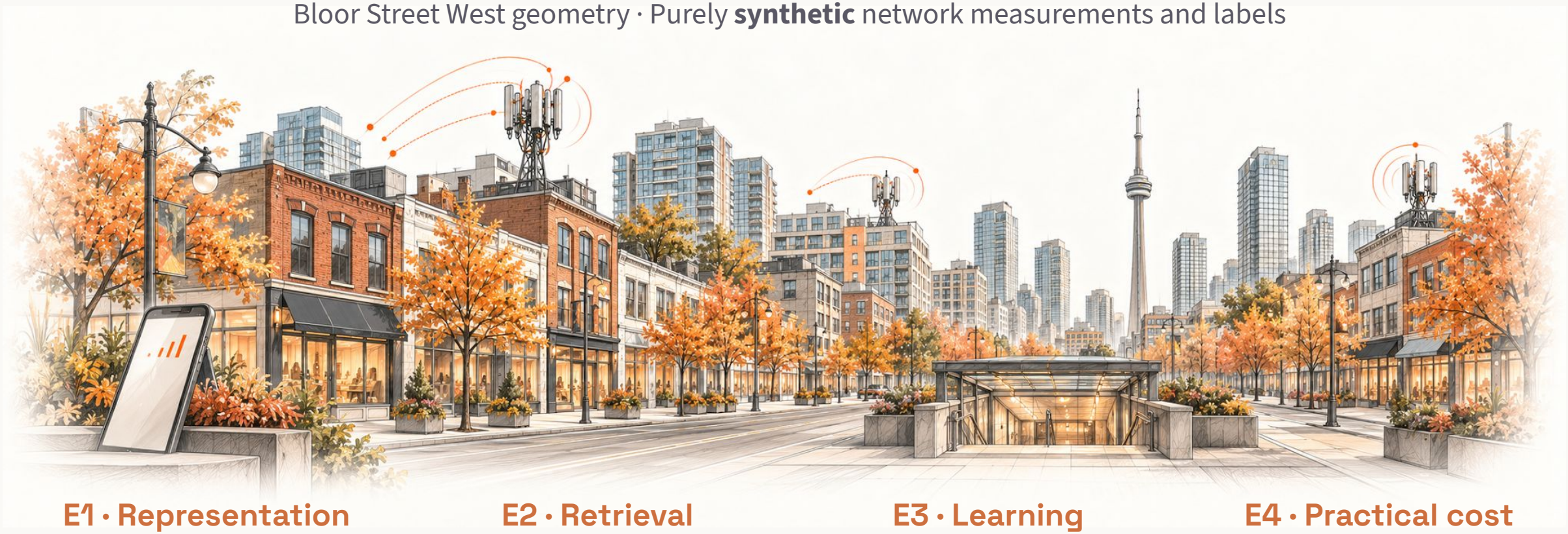
Compare it with all four prototypes.
Choose the class with the highest similarity.

Why normalize?

Unit-length episodes prevent raw magnitude from dominating, preserving direction and internal weights.
When we store hypervectors in LanceDB, we keep the class sum unnormalized. We normalize it only during cosine comparison.

Experimental design for the Bloor Street corridor study

Bloor Street West geometry · Purely **synthetic** network measurements and labels



E1 · Representation

Does the encoding preserve roles, order and connections?

E2 · Retrieval

Can we find similar incidents and trace their evidence?

E3 · Learning

Can class memory learn from a few labelled examples?

E4 · Practical cost

What does it cost to learn, keep models fresh and retain their state?

Only geographic coordinates are real: City of Toronto Open Data, Toronto Centreline (TCL).
All telecom observations are synthetic and for demonstration purposes only.

Hyperdimensional Computing Labs

E1: Does the encoding preserve roles, order and connections?

The same readings can mean different things when their roles, order or connections change.
We test whether the encoding keeps those cases distinct, so search and learning can tell them apart.

WHAT WE CHANGE	FULL ENCODING	RELEVANT FEATURE REMOVED
Swap the property meanings Good signal + bad link ↔ Bad signal + good link	≈ 0.039	≈ 1.0 No binding
Reverse the observation order Weakening signal ↔ Recovering signal	≈ 0.9	≈ 1.0 No order shifts
Change one network connection Same records, but a different serving connection	≈ 0.9	≈ 1.0 No graph context

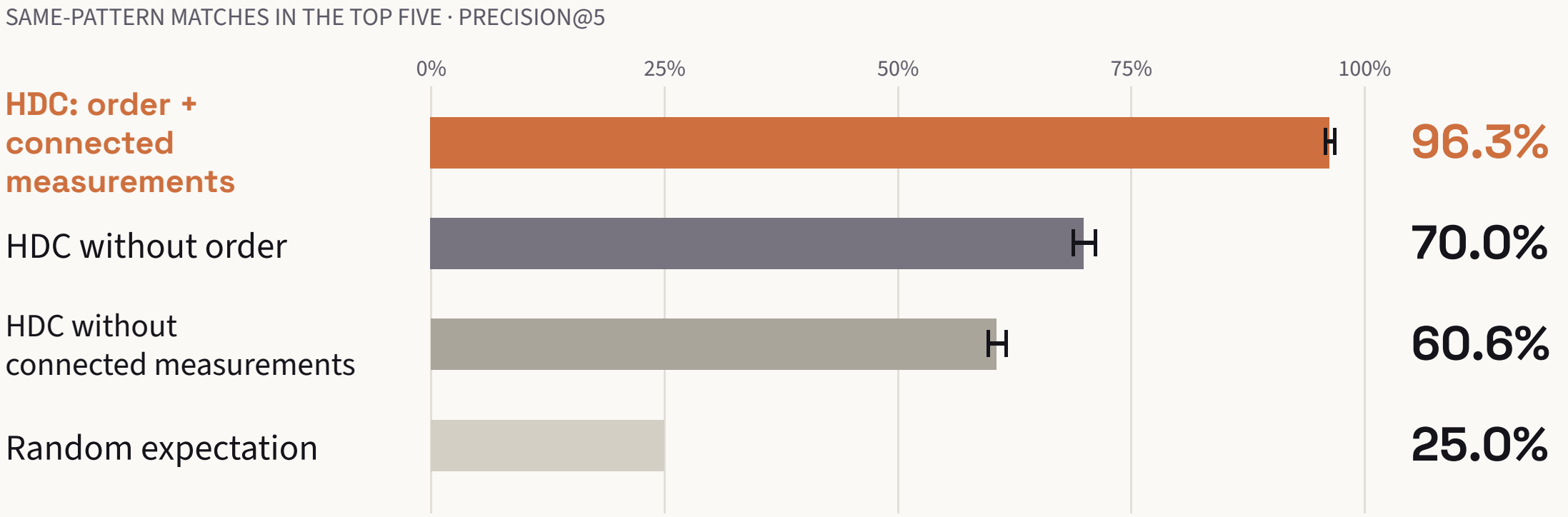
Cosine similarity: 1 means the same direction; lower values mean a changed representation

These checks test information preservation · Retrieval and learning performance come next

Hyperdimensional Computing Labs

E2: How well does HDC retrieve matching incident patterns?

- Rank earlier episodes by similarity and count same-pattern matches among the top five.
- Connected network measurements include cell load, backhaul packet loss and latency, and nearby phones' signal and packet loss.



Preserving order and connected measurements yields best results (96.3% match)

Order helps search distinguish deterioration from recovery, even when both contain the same readings.

Hyperdimensional Computing Labs

E2: Verify a retrieved match through its graph path

Cypher reaches the connected records behind each top match.
The run checks that their source IDs match the evidence used in its encoding.

QUERY · WEAKENING SIGNAL

−80.6 → −110.6 dBm

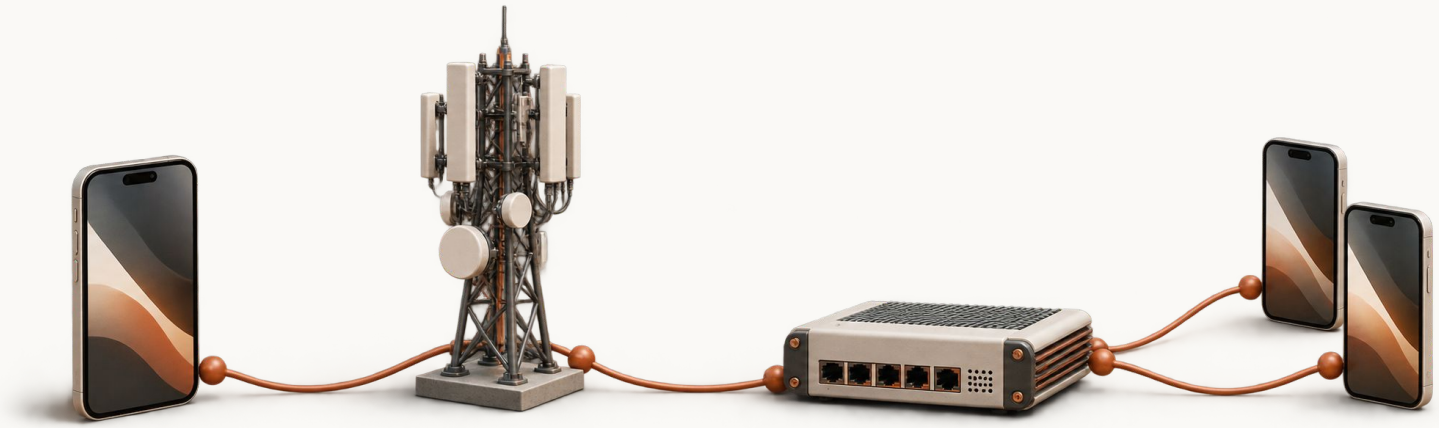
↓ similarity search

RETRIEVED EPISODE

Deteriorating radio

→

FOLLOW THE EVIDENCE TRAIL IN THE GRAPH



Phone
Signal

Serving cell
Load

Backhaul
Loss · latency

Peer phones
Signal · loss

CYPHER PATH EXCERPT · Full queries check observation times and peers

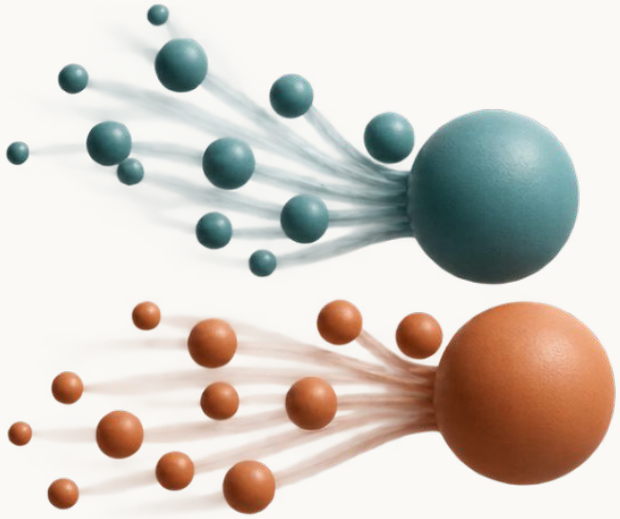
```
MATCH (e:Episode)-[:HAS_OBSERVATION]→(o:Observation)
      -[:SERVED_BY]→(c:Cell)-[b:BACKHAULED_BY]→(l:Link)
WHERE e.episode_id = $episode
```

Hyperdimensional Computing Labs

E3: How do three learning methods use labelled episodes?

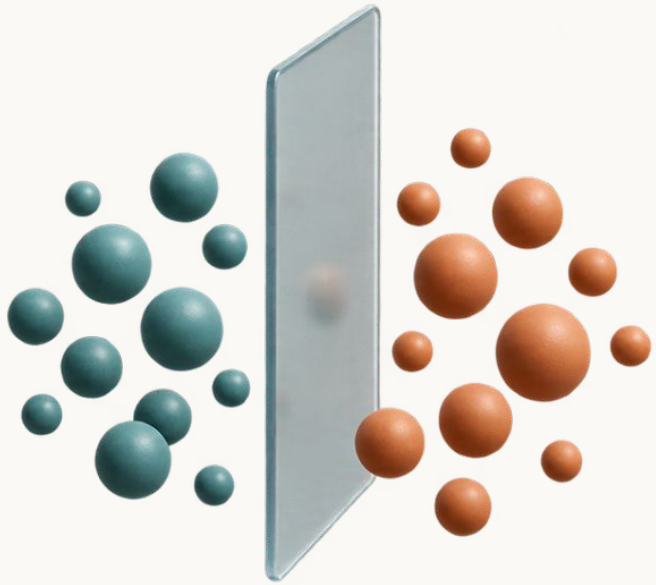
We increase the labelled training set and test how well each method classifies new incidents.

HDC · Class memories



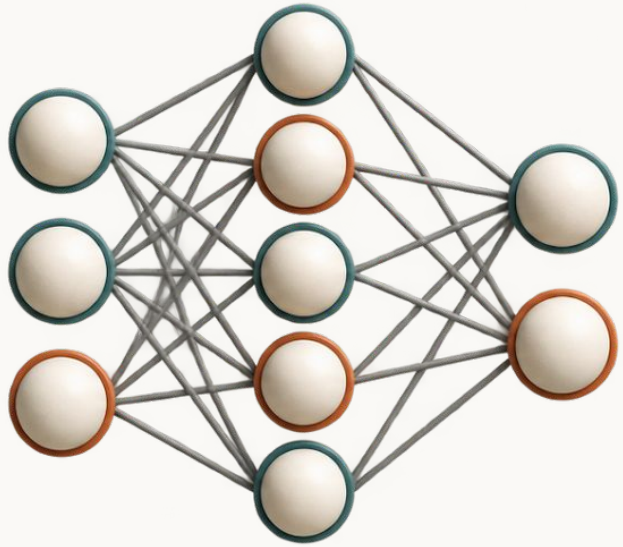
Add each labelled hypervector to its class **prototype**. Predict by similarity to class memories.
The encoder stays fixed.

Logistic regression



Fit weights that combine the 21 inputs into class scores. This learns linear boundaries between classes.

Neural network · MLP



Fit weights through one hidden layer to learn nonlinear relationships among the same 21 inputs.

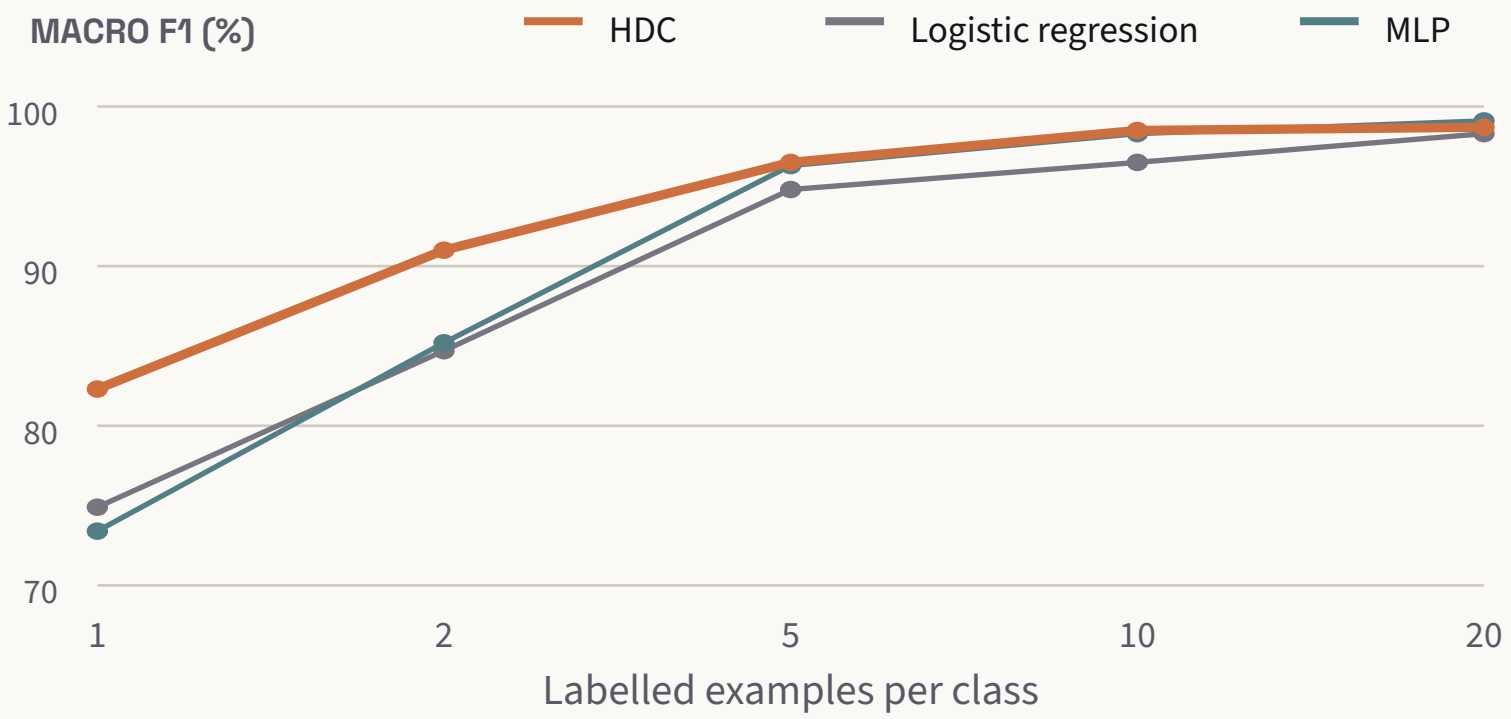
4, 8, 20, 40 or 80 labelled incidents in total

Evaluate on the same held-out episodes using macro F1, which weights all four classes equally.

Hyperdimensional Computing Labs

E3: HDC leads with four labelled training incidents

Four categories: 1 example per class means 4 labelled incidents in total



4 INCIDENTS · 1 PER CLASS

82.3% HDC

74.9% logistic regression
73.4% MLP

20 INCIDENTS · 5 PER CLASS

96.5% HDC

94.8% logistic regression
96.3% MLP

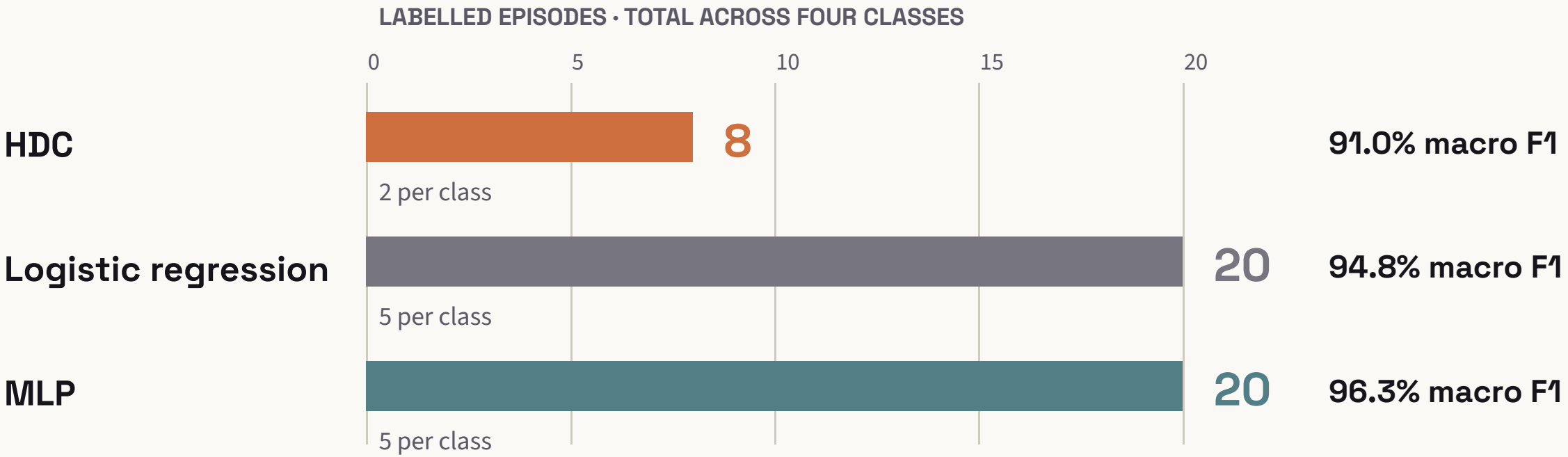
HDC's clearest lead is with the smallest training set

4 training incidents: one from each category. HDC reaches 82.3% macro F1.
20 training incidents: five from each category. All three methods reach about 95–96.5%.

Hyperdimensional Computing Labs

E3: HDC reaches 90% macro F1 with eight labelled episodes

First tested training budget at which each method crosses 90% on held-out episodes



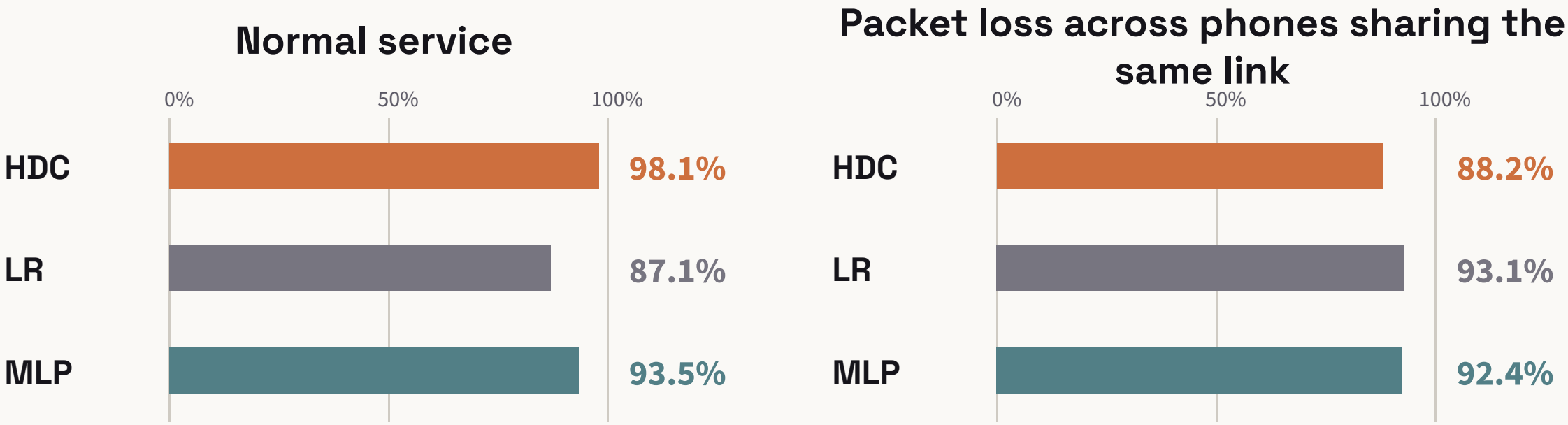
A strong starting point from a small labelled set

HDC crosses 90% using 40% of the labelled episodes used by LR and the MLP at this milestone.
In this study, **just two examples** of each pattern are enough to build useful class memories.

Hyperdimensional Computing Labs

E3: HDC recognizes 98.1% of normal-service episodes

Correct classifications within each pattern · Five labelled examples per class (20 total)



Share of episodes correctly classified · common 0–100% scale

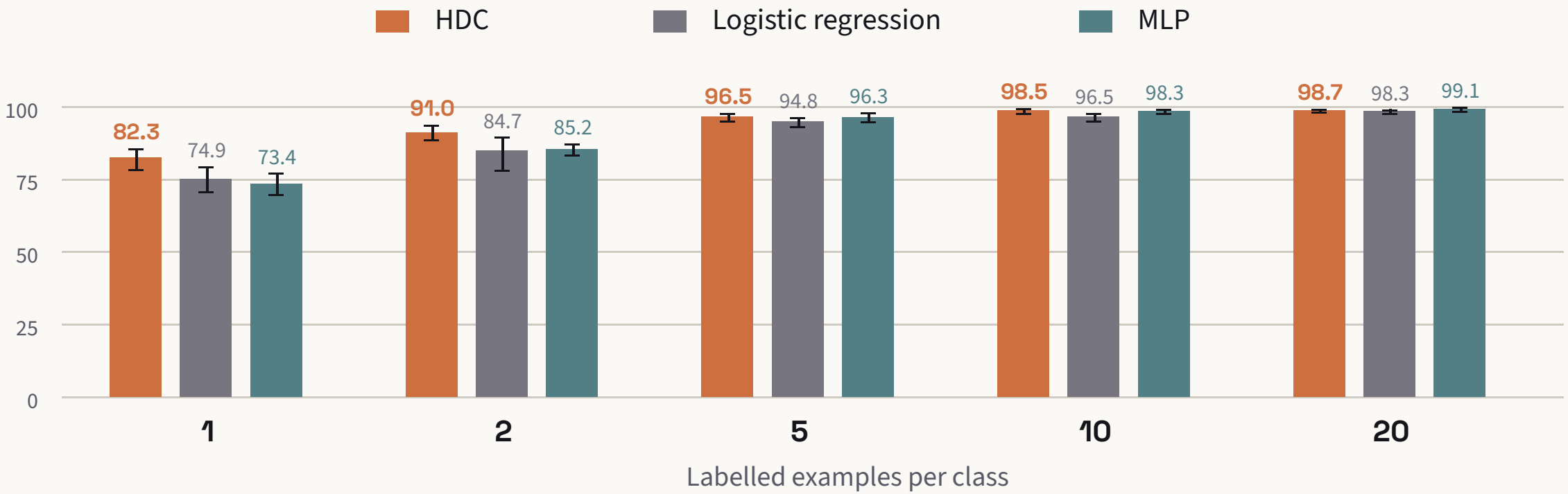
Next: test learning from mistakes, not just adding examples

- Current: every labelled episode contributes equally to its class memory, including distracting variation.
- Test with a fixed encoder: on errors, add to the correct memory and subtract from the predicted one.
- Test separately: learn weights for link and peer evidence. Neither change has been evaluated here.

Hyperdimensional Computing Labs

E3: Strong classification with direct memory updates

Mean macro F1 (%) across five label budgets · whiskers show 95% world-grouped intervals



HDC combines learning from few examples with direct memory updates

96.5% macro F1 at five examples per class. New labels update memory by addition—
no encoder retraining or replay of earlier examples.

Hyperdimensional Computing Labs

E4: What quality do we get for the compute cost?

Same training examples and independent test worlds · quality and initial learning time during training updates

4 labelled episodes

1 per class

HDC 82.3% F1 · 0.1 ms

LR 74.9% F1 · 1.9 ms

MLP 73.4% F1 · 10.2 ms

20 labelled episodes

5 per class

HDC 96.5% F1 · 0.4 ms

LR 94.8% F1 · 2.5 ms

MLP 96.3% F1 · 27.8 ms

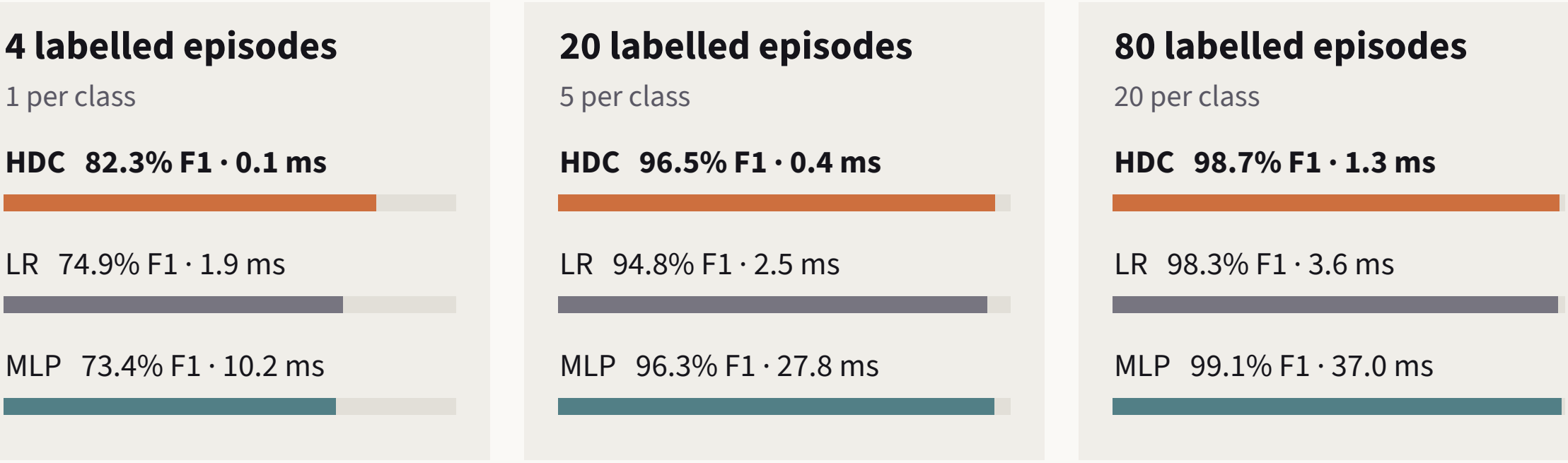
80 labelled episodes

20 per class

HDC 98.7% F1 · 1.3 ms

LR 98.3% F1 · 3.6 ms

MLP 99.1% F1 · 37.0 ms



Bars: macro F1 on a common 0–100% scale. Times: initial learning only, with inputs already encoded.

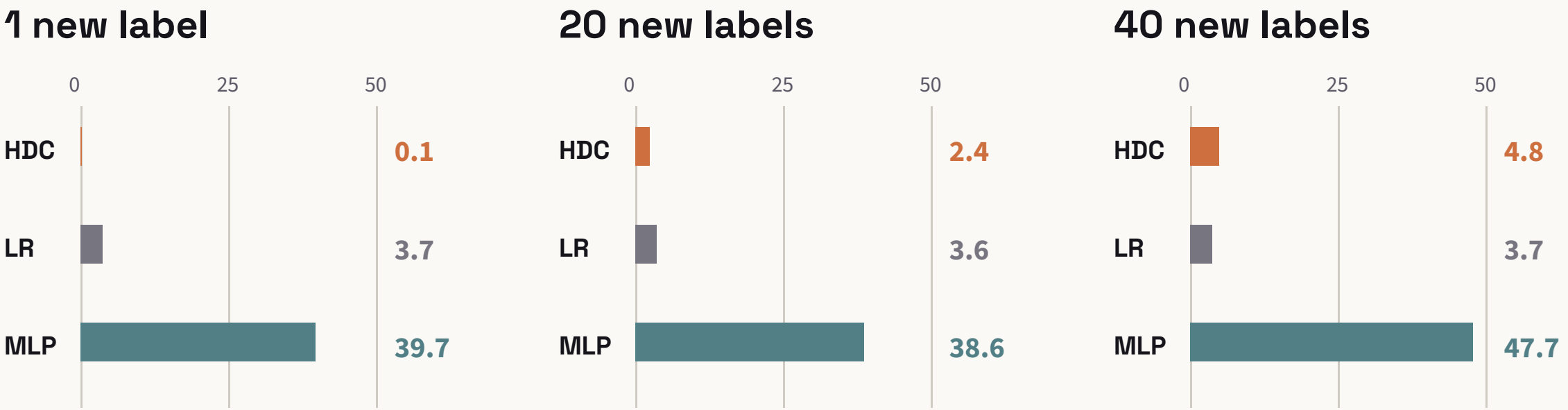
At 80 labels, all three score above 98%. What does the next label cost?

Next slide: start from those 80 labels and time updates. Post-update test quality was not measured.

Hyperdimensional Computing Labs

E4: HDC can learn effectively via frequent, small updates

Time to prepare inputs, learn and predict · same 80 starting reviews · milliseconds on one CPU thread



Update as labels arrive, or wait and refit a batch?

One-label updates take 0.1 ms for HDC, 3.7 ms for LR and 39.7 ms for MLP.
At 40 labels, LR's single refit is faster: 3.7 ms versus 4.8 ms for HDC. Incremental LR/MLP was not tested.

Hyperdimensional Computing Labs

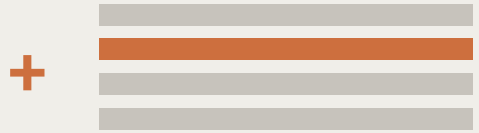
E4: Add to memory, or relearn the model's weights?

New labels change different parts of the system

HDC

Vector arithmetic

New labelled episode → **Fixed encoder** Build a hypervector → ADD TO CLASS MEMORY



Same slots; updated values

LR / MLP

Batch refit in this study

Earlier examples + New labelled episodes → **Fixed input features** Build a numeric vector → **Learned model weights** Refit on earlier and new examples

HDC's class memory stays fixed in size as labels accumulate.

Hyperdimensional Computing Labs

HDC connects structured similarity with adaptable memory

Four results to carry into your own experiments

E1 · REPRESENTATION

Meaning survives encoding

Roles, network connections and observation order shape the hypervector.

E2 · RETRIEVAL

96.3% matching results

Similarity finds comparable episodes; the graph provides the evidence trail.

E3 · LEARNING

82.3% F1 from four labels

One example per class provides a useful initial memory; more labels improve it.

E4 · UPDATES

0.1 ms for one new label

Add to class memory directly, with a fixed encoder and no full model refit.

Next, test the same ideas against operational complexity

Real telemetry · missing measurements · class imbalance · changing patterns · complete application costs

Hyperdimensional Computing Labs

Where else would structured similarity help?

HDC has numerous applications, from physiological and speech data, to molecular structure and temporal and geospatial analyses



EEG seizure detection



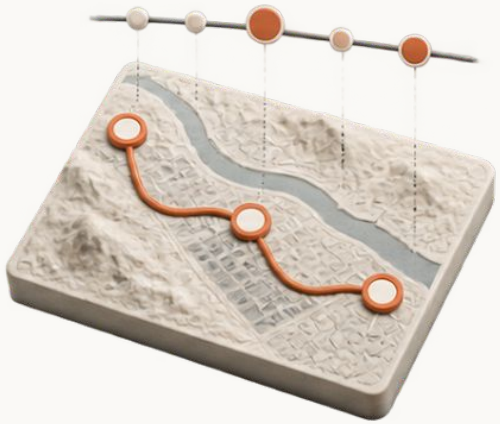
Wearable gesture recognition



Spoken-letter classification



Molecule / protein classification



Temporal and geospatial analysis

[Explore the study, results and implementation]

HDC turns structured observations into reusable memory

Model what matters, compare whole episodes and learn as labels arrive

MODEL

Choose what similarity means

Bind roles to values, bundle facts and preserve order. Nearby numeric levels remain similar.

REUSE

One hypervector, two uses

Search for comparable episodes and classify new ones using the same fixed-size representation.

LEARN

Build memory from examples

Add labelled episodes to class sums as they arrive. Our encoder stays fixed; no backpropagation is needed.

INVESTIGATE

Keep the graph in the loop

HDC finds similar patterns. The graph provides exact connections and a traceable path to source evidence.

HDC complements graphs with similarity and adaptable class memory

Its modelling advantage is compositional: you decide which facts, relationships and sequences should shape similarity.

Keep exploring HDC

Code & results

github.com/hyperdimensionalcomputingai/telecom-incident-triage

HDC blog

hyperdimensionalcomputing.ai/blog/



David Hughes
Founder, HDC Labs
Principal AI Solutions Architect



Prashanth Rao
Cofounder, HDC Labs
AI Engineering & Research Lead

What we're building

Systems combining HDC and graphs for contextual retrieval, agent memory and learning from new examples.

Meet the team: hyperdimensionalcomputing.ai/people/